DETAILED LITERATURE REVIEW

Energy-Aware Adaptive CPU Scheduling for Heterogeneous Multicore Edge Devices Using Deep Reinforcement Learning

2020–2026 | Repository-aligned literature review and implementation assessment

Literature: 15 Oct 2026 | Repository review: 16 Oct 2026 | 25 references | 38 pages

Abstract

Energy-aware scheduling for heterogeneous edge CPUs requires coordinated resource selection, power management, timing analysis, and credible evaluation. This review synthesises 2020-2026 research and aligns it with the energy-aware-drl-scheduler repository at a pinned commit. The project contains a Gymnasium aggregate-state environment, a saved PPO model, a C inference scaffold, and a Streamlit dashboard. Inspection distinguishes these artifacts from a fully integrated learned hardware scheduler: the reviewed dashboard runs a C simulator with explicit dummy weights, not the saved PPO model, and does not actuate CPU frequency or measure board energy. Local C checks show that the pinned actor always chooses the lowest-power action; its 92% improvement follows from fixed power ratios, with unequal served work and silently clipped backlog. The final revision rebuilds the C engine, adds work accounting and a Python-C parity test, and re-measures on equal served work. The corrected figure remains a property of the assumed power model, not a hardware saving. The literature supplies a development path through temporal representation, contention-aware service, invocation accounting, actor-export verification, and realistic hardware constraints. The review proposes reproducible training, explicit time and work units, stronger baselines, and staged device validation, while separating implemented features, published findings, local checks, and future claims.

Keywords: heterogeneous multicore; edge computing; CPU scheduling; DVFS; deep reinforcement learning; partial observability; thermal management.

Project: https://github.com/AdityaRezan/energy-aware-drl-scheduler

PROJECT TEAM

Aditya Rajeshwar Karn - 24BEL0087 - aditya.rajeshwar2024@vitstudent.ac.in

Yuvraj Singh - 24BEL0057 - yuvraj.singh2024@vitstudent.ac.in

Aditya Singh - 24BEL0008 - aditya.singh2024c@vitstudent.ac.in

Atharva Sivaraman - 24BEL0082 - atharva.sivaraman2024@vitstudent.ac.in

Yash Jain - 24BEL0035 - yash.jain2024a@vitstudent.ac.in

Final submission: 26 October 2026 | Supplementary sections 24-29 and Appendices A-B added

Contents and Reading Guide

The review is organized into project scope, chronological research evidence, and an implementation-

1. Review Scope and Repository Provenance 3
2. Research Problem and Implemented Scope 4
3. Energy Semantics and Simulator Accounting 5
4. Hardware Realism and the Project's Execution Paths 6
5. The 2020 Foundations: Thermal Control and Simulation 7
6. Fairness and Shared-Resource Contention, 2021-2022 8
7. Runtime Power Learning and Scalable Action Selection 9
8. Thermal-Aware DRL and the Limits of Neural Transfer 10
9. Temporal Edge Governors and the Offloading Boundary 11
10. DAG Representation and Multiobjective Scheduling in 2024 12
11. Multi-Task, Multi-Deadline DVFS: 2024-2025 13
12. FiDRL and the Economics of Controller Invocation 14
13. Integrated Heuristics and Complex Learning Frameworks 15
14. Joint Heterogeneous Control on Real Edge Hardware 16
15. Partial Observability Becomes Explicit in 2026 17
16. HiDVFS: Hierarchy, Thermal Feedback, and Runtime Screening 18
17. Broader 2026 Evidence and Cross-Domain Transfer 19
18. Repository Architecture and Evidence Map 20
19. Implemented State, Actions, Reward, and Dynamics 21
20. Local C Checks and Interpretation of Reported Savings 22
21. Repository-Aligned Architecture and Revision Priorities 23
22. Evaluation Plan for the Supplied Repository 24
23. Synthesis, Claim Boundaries, and Project Deliverables 25
24. Extended Review Methodology and Evidence Grading 26
25. Comparative Evidence Synthesis Across 25 Sources 28
26. Post-Review Remediation and Re-Measurement 30
27. Statistical Protocol and Reporting Standards 32
28. Team Contributions and Project Governance 33
29. Limitations, Ethics, and Claim Boundaries 34
Appendix A. Reproducible Artefact Manifest 35
Appendix B. Glossary, Symbols, and Abbreviations 35
Annotated references [1]-[8] 36
Annotated references [9]-[16] 37
Annotated references [17]-[25] 38

How to use this review

For technical grounding, read pages 3-6. For the closest implementation studies, prioritise pages 11-18. For repository architecture and code checks, use pages 20-25. For methodology, evidence grading, remediation and the statistical protocol, read the supplementary pages 26-34. The artefact manifest and glossary are on page 35; annotated references occupy pages 36-38. Publication status and source roles are identified in the annotated bibliography.

1. Review Scope and Repository Provenance

1.1 Literature window and project alignment

This review examines research dated 2020–2026, with the literature search retained through 15 October 2026 and a repository inspection completed on 16 October 2026. It now incorporates AdityaRezan/energy-aware-drl-scheduler [25] as the implementation under study. The repository is a software artifact, not a research paper or independent validation of its own claims. The reference set contains 25 entries: 22 journal or conference papers, including three reviews, one master's thesis, one preprint, and this source-code repository. Published findings, observed implementation details, local checks, and future recommendations are distinguished throughout.

The target research topic remains energy-aware control on heterogeneous edge CPUs. The actual repository, however, presently implements an aggregate simulation rather than an operating-system task dispatcher. That distinction changes the research positioning: the project is a useful prototype for studying PPO-oriented control and a C execution path, but the inspected code does not demonstrate physical CPU affinity changes, frequency actuation, measured board energy, or hard real-time guarantees. The literature is used to identify what this prototype represents and which extensions are needed to substantiate stronger claims.

1.2 Reproducible inspection boundary

The inspected snapshot is commit 2093bec5206c051cd4d806b2f1bd8a58634e42e1. Repository statements in this document refer to that version, not to later changes on the main branch. The README, Python environment, Streamlit application, C implementation, weights header, build file, dependency list, saved-model archive metadata, and benchmark image were inspected. A local copy of the C program was compiled and run; a separate review harness measured action selection and accounting behavior. These checks were performed in a sandbox and are not measurements of the target hardware [25].

Only plain-text metadata from the saved PPO archive was read; its serialized Python objects and neural parameters were not loaded for execution. Consequently, no claim is made that the stored PPO policy was retrained, reproduced, or shown equivalent to the C policy. The current repository does not include training, export, or benchmark-generation scripts that establish that chain. No changes were pushed to the repository. Its URL and pinned version are included in the bibliography for traceable future comparison.

1.3 Evidence standards

The literature selection remains a focused narrative synthesis, not exhaustive systematic coverage. Some papers were accessible only through abstracts or previews, and their reported measurements were not independently reproduced. By contrast, repository-specific findings identify concrete files or local checks. This separation prevents published DRL results from being inadvertently attributed to the student implementation. It also makes limitations actionable: a missing interface, uncalibrated metric, or disconnected policy artifact can be addressed by an explicit engineering and validation task rather than hidden behind a general claim of intelligent scheduling.

2. Research Problem and Implemented Scope

2.1 The intended problem

Energy-aware heterogeneous CPU scheduling involves deciding which task should execute, on which core, and at which supported operating point while preserving declared service requirements. True heterogeneous placement requires identifying participating core types, their capacities, and any shared frequency domains. Task-level evaluation additionally needs release times, execution demand, completions, and, where relevant, deadlines or dependencies. Fairness-aware scheduling and joint cluster–frequency control provide examples of these richer requirements [2], [13], [19].

The project title expresses that research direction, but the present environment is narrower. EdgeSchedulerEnv maintains one normalized queue-load value, one normalized thermal value, and one binary core-class indicator. It does not create identifiable jobs, maintain per-core queues, track multiple simultaneously running cores, or assign individual tasks to them. Four actions select predefined power, processing-capacity, and temperature-increment coefficients. The indicator is set to one for actions two and three, and zero otherwise. This approximates two resource classes without constructing a physical multicore topology [25, edge_env.py:10–45].

2.2 What “adaptive” means in this implementation

The repository contains a Stable-Baselines3 PPO archive and a separate C inference function. A saved policy can, in principle, adapt its decisions to changing observations without updating its parameters. Online learning is a different capability and is not implemented in the dashboard’s C execution path. Furthermore, the current weights.h explicitly contains dummy compiler-test weights; the C function therefore does not demonstrate the behavior of the saved PPO archive. The precise implication is analyzed in Section 20 rather than assuming that a DRL label establishes a trained decision process [25].

The Streamlit application allows peak-power scale, initial queue load, and simulation duration to be changed. These controls adjust simulator parameters, not the hardware architecture of the host CPU. Presets named after a Raspberry Pi, mobile big.LITTLE device, or desktop processor change the default power value; they do not load calibrated core tables, frequency ranges, or board-specific thermal models. “Custom hardware simulation” is therefore an interface label for parameterized simulation, not evidence of hardware portability.

2.3 An appropriate current research question

A defensible immediate question is whether a reproducibly trained PPO policy improves a synthetic power–queue–thermal trade-off over strong baselines under the same arrival process. A later question is whether an exported, verified actor preserves that behavior in C. Actual heterogeneous CPU scheduling and net device-energy savings require another validation stage. Keeping these questions separate preserves the project’s educational and experimental value while avoiding unsupported claims of kernel integration, physical DVFS, multicore task dispatch, or guaranteed deadline satisfaction.

3. Energy Semantics and Simulator Accounting

3.1 Distinguishing modeled power from measured energy

Energy is accumulated power over physical time. A power model can support an energy estimate only when its units and interval duration are defined. The Python environment assigns action-dependent values of 0.2, 0.5, 1.2, and 2.5 to the field named power. The C simulator instead multiplies a user-specified peak-power scale by ratios 0.08, 0.20, 0.48, and 1.00. At a scale of 2.5, the numerical coefficients agree; at the dashboard default of 15, the C values are six times the Python values [25].

The C implementation adds each action's modeled power directly to total_power, then prints the total as "Joules." No explicit step duration is defined or multiplied into that sum. The label is physically valid only after an interval assumption, such as one second per step, is declared and the coefficients are accepted as watts. This review therefore calls the existing output an accumulated modeled-power value. It is not measured electrical energy, CPU package energy, or proof of board-level savings [25, main.c:61–79,110–115].

3.2 Why the power-profile slider does not validate portability

The peak-power input affects the reported numerical power scale, but not queue transitions, thermal transitions, or the policy observations. For a fixed action sequence, changing the scale multiplies both comparison totals and leaves their percentage difference unchanged. The current policy also receives no peak-power input. Consequently, obtaining similar percentage savings under 5 W, 15 W, or 65 W presets does not show that a policy learned different hardware behavior. It follows from the shared scaling rule, not from experimental calibration of three devices [25].

Real devices can have nonproportional power curves, shared rails, leakage changes, and different operating-point efficiency. The characterization and overhead lessons in [3], [9], and [13] therefore remain necessary. A revised simulator should define a timestep, documented power units, and energy as the sum of power multiplied by elapsed time. It should distinguish CPU-domain estimates from whole-board measurements and state whether monitoring, inference, and transitions are included.

3.3 Matched work and conservation

Energy comparisons must also account for completed work. Both simulators clip queue load to the interval zero to one without reporting the excess removed at the upper bound. This can hide overload and discard demand silently. At the lower bound, processing capacity can exceed available demand without a completed-work counter. An explicit conservation equation should track arrivals, served work, remaining backlog, and rejected work. Without that accounting, a low-power controller can appear efficient simply because it serves less work. Section 20 quantifies this issue for the current C path; it must be resolved before interpreting the displayed improvement as a service-equivalent gain.

4. Hardware Realism and the Project's Execution Paths

4.1 What the dashboard actually invokes

The Streamlit application compiles main.c with GCC and executes the resulting scheduler binary, passing peak-power scale, initial queue load, and step count as command-line arguments. It parses the printed totals and displays a two-bar comparison. This is a functioning interface between a dashboard and a native simulator. It is not a call to Linux CPUFreq, an actual performance-governor experiment, or a CPU-affinity controller. The C routine named step_hardware updates floating-point simulation variables and samples a synthetic arrival; it does not communicate with hardware control interfaces [25, app.py:43–76; main.c:61–79].

A separate path consists of edge_env.py and the saved PPO archive. The current app.py imports neither the environment nor Stable-Baselines3 and does not load that archive. Therefore, the dashboard results cannot be attributed to the saved model merely because both are stored in the same repository. The missing connection is an inspectable export and equivalence-validation pipeline, not a cosmetic documentation change.

4.2 The baseline is synthetic

The comparison loop resets the C pseudorandom generator to the same seed used for the candidate loop, which gives both paths the same arrival sequence within that C run. This is a useful paired-design feature. However, the baseline always selects action three in the simulator. It does not switch the machine running the experiment into Linux performance mode or measure that machine's consumption. The appropriate label is "synthetic always-maximum configuration," unless a future hardware experiment implements and measures an actual governor [25, main.c:94–108].

Likewise, the normalized thermal variable is not a calibrated temperature in degrees Celsius. Higher modeled temperature does not reduce available processing capacity or trigger hardware throttling in either implementation. The Python reward applies a threshold penalty, whereas the C benchmark does not compute reward at all. Consequently, "thermal penalty" or "modeled thermal state" is more accurate than a claim of implemented thermal-throttling control.

4.3 A staged route toward real deployment

Literature on thermal migration [1], practical governors [6], and heterogeneous joint control [13] supports a staged approach. First, make the simulator and actor export reproducible. Second, define legal operating-point and affinity interfaces for one selected board. Third, calibrate capacity, power, transition latency, and thermal behavior. Finally, compare service and energy on the device while retaining native protection. Until those stages are completed, the document should describe an application-level simulation prototype. A C implementation can be efficient and educational without being misrepresented as an operating-system scheduler or a verified hardware controller.

5. The 2020 Foundations: Thermal Control and Simulation

5.1 Kim et al.: using heterogeneity under thermal pressure

Kim et al. [1] propose an adaptive thermal-management framework for heterogeneous multicore processors. The framework considers migration between big cores and migration between big and small cores, rather than relying exclusively on frequency reduction when a core becomes hot. If a cooler big core is available, migration can preserve access to high-performance execution. Otherwise, the framework compares a big-to-small migration choice with a DVFS-based response. Its embedded-board evaluation provides direct evidence that placement and thermal decisions are coupled on real hardware.

The importance for a DRL literature review is conceptual and comparative. Thermal state changes the set of useful scheduling actions, and hardware heterogeneity can be exploited without neural learning. A new agent should therefore be compared with a competent thermal-aware heuristic rather than a temperature-oblivious placement rule alone. The original study addresses performance under thermal constraints; it should not be recast as proof that every migration minimizes total energy. Application phase, migration cost, and remaining work affect whether that intervention is beneficial.

5.2 DS3: a controlled environment, not a deployment result

Arda et al. [18] introduce DS3, a system-level simulation framework for domain-specific heterogeneous systems-on-chip. It supports exploration of scheduling and power–thermal management and demonstrates applications from wireless communications and radar processing. Its contribution is infrastructure for examining interactions among workloads, processing elements, and runtime management. Such infrastructure is valuable because repeated hardware experimentation can be slow, expensive, and sensitive to uncontrollable background conditions.

However, a simulator is an experimental instrument rather than independent confirmation of a learned policy. Its execution-time estimates, communication costs, thermal behavior, and scheduling-event semantics determine what an agent can exploit. A policy trained in an unrealistically inexpensive control loop can appear effective even when its real inference overhead would dominate. Similarly, a deterministic simulator may reward precise timing choices that become brittle under interrupts, variable memory latency, and sensor noise. Calibration should therefore compare distributions and transitions, not only average task duration.

5.3 Implications for the proposed research

Together, [1] and [18] establish a practical sequence: characterize the physical device, reproduce its main behaviors in an experimental model, and then test control alternatives. The learning system should not define the problem before the hardware constraints are understood. For the supplied project [25], four abstract actions are a starting model, not a measured operating-point table. Domain-level controls and compatibility rules remain future work. Thermal disturbances and transition delays should be varied during evaluation to reveal sensitivity. These requirements remain relevant through 2026; newer neural architectures do not remove the need for accurate system modeling and non-learning control baselines.

6. Fairness and Shared-Resource Contention, 2021–2022

6.1 Energy efficiency with a service-distribution constraint

Salami, Noori, and Naghibzadeh [2] investigate fairness-aware energy-efficient scheduling on heterogeneous multicore processors. Their framework uses DVFS and is evaluated on a real heterogeneous platform. Its relevance is that heterogeneity changes both efficiency and access to performance. If a scheduler repeatedly assigns favored tasks to faster cores, overall throughput can improve while another task's slowdown becomes unacceptable. A learning objective based only on total throughput or mean delay may reproduce this behavior unless fairness is made visible.

Fairness is not synonymous with equal CPU time. Different tasks can require different amounts of execution capacity, and equal time on unlike cores does not deliver equal progress. A study must specify whether it targets proportional service, bounded slowdown, priority-weighted fairness, or another application-appropriate definition. The metric should correspond to the intended service contract. For example, deadline-constrained control tasks and background analytics may legitimately receive different treatment, but a low-priority task should not disappear from evaluation simply because it completes infrequently.

6.2 Contention changes the task–core relationship

The later study by the same authors [19] explicitly incorporates shared-resource contention alongside fairness and energy efficiency. Its real-platform evaluation underlines that task performance cannot always be predicted independently. Two tasks can interfere through shared caches, memory bandwidth, or other platform resources. A core that is efficient for an isolated task may no longer be the best choice when colocated tasks compete for those resources. Thus, a scheduler that learns only task identity and core type risks overlooking an important source of nonstationarity.

This creates a state-design trade-off. More hardware counters may reveal contention but impose monitoring cost and reduce portability. Minimal utilization-based observations are inexpensive but can conflate compute demand with memory stalls. A carefully designed experiment should compare a lightweight history representation with a modest set of contention indicators and report the marginal benefit of each. The argument is not that every counter is necessary, but that measurement choices should be justified by an observed ambiguity in the control problem.

6.3 Consequences for DRL evaluation

A new controller should report per-task or per-class performance in addition to system totals. Paired workloads can test whether energy savings persist when CPU-bound and memory-bound tasks run concurrently. The same admission policy and task priorities should be used across baselines. If the learner is allowed to delay or reject jobs while the comparator is not, the energy comparison is confounded. Studies [2] and [19] therefore supply more than historical background: they define substantive service and interference baselines against which a heterogeneous edge DRL scheduler must justify its contribution.

7. Runtime Power Learning and Scalable Action Selection

7.1 Wang et al.: phase-aware power and uncore control

Wang et al. [3] propose reinforcement-learning-based online power management combining power capping with uncore-frequency control. The framework learns relationships between program behavior, hardware configuration, energy, and execution time. Its evaluation includes 11 parallel programs not used during training on two computing clusters. The authors report average energy reduction of 12% with less than 3% slowdown. These results support learning across program phases and indicate that resources outside the CPU execution cores can influence the best power configuration.

The evidence is relevant but not architecturally identical to heterogeneous edge scheduling. Power capping can indirectly influence processor behavior, while an edge platform may expose different knobs and energy sensors. The study does not establish that choosing between big and LITTLE cores will achieve the same gain. The transferable lesson is to identify the controllable resource that actually limits energy efficiency, rather than assuming CPU frequency is always the dominant variable. A proposed scheduler should also test whether its learned policy generalizes to unseen applications, not just unseen samples of a familiar program.

7.2 Partitioned actions in large heterogeneous systems

Jingbo Li et al. [21] study energy-aware task scheduling using PPO in large-scale heterogeneous systems. Their approach uses a SPECpower-based energy model, partition-based resource selection, and an autoencoder for high-dimensional observations. Simulations across different system scales support the idea that representation and action decomposition can make learning more tractable. This is an architectural contribution to scheduling research, although a server-level power model should not be imported into an edge-CPU setting without calibration.

Partitioning reduces the immediate choice set by selecting a resource group before selecting an individual resource. A similar idea could separate task choice, CPU cluster choice, and operating-point choice. However, decomposing the action does not remove coupling: the best frequency may depend on the task and the cluster already selected. An independent policy head for each variable can miss this relationship. Conditional or hierarchical action selection is therefore a design option, not a guarantee of superior performance.

7.3 What these studies establish collectively

The two studies illustrate distinct scaling problems: changing program phases over time and increasing resource-choice dimensionality. A compact edge controller may need both temporal information and structured actions, but it should not inherit a large architecture solely because large-scale systems require one. The appropriate comparison is between simple and structured policies under a fixed observation budget and comparable training effort. Gains should be decomposed into those arising from additional information, a better action model, and the learning algorithm itself. Without that separation, improved results can be incorrectly attributed to neural sophistication rather than a more favorable problem representation.

8. Thermal-Aware DRL and the Limits of Neural Transfer

8.1 Q-scheduler: energy and temperature together

Mohammadi and Beitollahi [4] propose Q-scheduler, a deep Q-learning approach for dispatching tasks in a real-time homogeneous multiprocessor embedded setting. The method uses task and processor information, including utilization and thermal characteristics, to guide allocation. It contributes evidence that temperature can be integrated into learned scheduling rather than handled only by a separate emergency mechanism. Temperature matters because it reflects recent activity and can constrain the performance available to future tasks.

Its homogeneous architecture is an important boundary. A thermal-aware policy that chooses among similar processors does not necessarily learn the performance–energy trade-off between different core types. Heterogeneous scheduling requires evaluating whether a task should occupy a faster core, not merely a cooler one. The study's reported objectives also should not be confused with a general proof of real-time schedulability. The appropriate interpretation is a learned allocation strategy evaluated under the paper's workload and system assumptions.

8.2 Sung and Ryu: practical utility depends on timescale

Sung and Ryu [5] investigate neural scheduling for systems-on-chip and identify challenges that prevent direct transfer from cluster scheduling. These include heterogeneity, variable action sets caused by incoming jobs, and the relationship between processing-element switching delay and computation time. Their interaction-matching approach addresses aspects of the learning problem, while the broader result is a warning that implementation overhead can determine whether a neural policy is useful at all.

The project's C implementation [25] has no measured inference timing or invocation-energy result. Compilation alone therefore cannot establish the low-overhead advantage needed for short tasks. Longer application phases can tolerate more analysis. Consequently, a publication should report absolute decision latency and its proportion of the control interval, not only a percentage reduction relative to another scheduler. Worst-case or high-percentile control latency also matters because occasional slow decisions can damage timing behavior even if average inference is inexpensive.

8.3 Combining thermal awareness with feasibility

The synthesis is not to discard DRL, but to constrain its role to a measured decision budget. A policy may select placements at application boundaries while conventional mechanisms handle finer-grained dispatch. Alternatively, a fast cached decision can be used between less frequent policy updates. These are proposed design choices, not claims that [4] or [5] already validates the same architecture. Each requires testing under realistic interference. Thermal control, scheduling latency, and policy complexity must be co-designed because they interact: the controller's own execution can increase load and heat while simultaneously attempting to reduce them.

9. Temporal Edge Governors and the Offloading Boundary

9.1 Zhou and Lin: history as an economical state representation

Zhou and Lin [6] develop a temporal-encoding-based DRL governor for periodic soft-real-time applications and implement it on an NVIDIA Jetson Nano. Their approach addresses the ambiguity of instantaneous utilization: two observations with the same utilization may correspond to different execution phases and different amounts of work remaining before a deadline. Encoding temporal progress gives the learner information relevant to future decisions without requiring a detailed explicit application model. The paper reports additional energy savings of 3%–11% over Ondemand on the evaluated MiBench workloads, depending on available slack.

The deployment contribution is important. Quantized in-kernel inference shows that a learned governor need not require an excessively large inference engine. However, implementation feasibility on the tested platform does not establish heterogeneous CPU placement. A CPU-plus-GPU board is not equivalent to a big.LITTLE CPU scheduling experiment. The periodic workload also provides repeated structure that may make phase learning easier than under sporadic arrivals or strong interference. These are scope conditions, not reasons to dismiss the result.

9.2 What temporal encoding can and cannot infer

History can distinguish recurring phases, but it does not make future arrivals observable or eliminate uncertainty in execution demand. The relevant question is whether the encoded history improves the control decision at acceptable monitoring cost. An ablation should compare instantaneous observations, compact temporal features, and a recurrent model under the same task traces. If improvement comes primarily from including deadline progress, it should not be attributed entirely to the choice of neural architecture.

The approach also raises portability questions. A representation based on progress within a known period may need modification for irregular releases or overlapping tasks. Normalizing elapsed time by a deadline can help accommodate different timing targets, but changing cache behavior or task composition may still alter the relation between utilization and remaining work. The later multi-task and partially observable studies [8], [14] make these questions increasingly explicit.

9.3 Adjacent evidence from deadline-aware offloading

Panda, Lin, and Zhou [22] combine DRL, DVFS, and application-deadline-aware offloading for time-critical IoT applications. The work is relevant because local execution speed and remote execution decisions share a timing budget. Nevertheless, offloading adds transmission delay, communication energy, and remote service assumptions. Its results cannot be used as direct proof that local CPU scheduling alone delivers comparable savings. For the present research, it provides a model of coordinated decisions under deadlines, while also illustrating why the system boundary must be fixed before comparing energy outcomes. Removing communication from the objective changes the problem being solved.

10. DAG Representation and Multiobjective Scheduling in 2024

10.1 Graph-attention actor–critic scheduling

Peng et al. [7] propose a dynamic scheduler that combines actor–critic learning with graph attention for real-time DAG tasks. A graph representation is appropriate because a task's importance depends on its predecessors, successors, and contribution to future critical paths, not only its own execution cost. The method jointly considers processor allocation and voltage/frequency selection. Its comparison with an optimal method in simulation supports efficient approximation within the stated model rather than global optimality in unrestricted systems.

Graph models address a genuine representation challenge: the number of tasks and their dependencies can change, making a fixed concatenated vector difficult to scale. However, encoding structure is not sufficient to obtain valid schedules. Readiness, processor compatibility, and resource occupancy still need enforcement. An action mask can exclude tasks with incomplete predecessors, but predicted deadline feasibility requires additional information. The costs of graph construction and inference should also be measured, especially when small tasks trigger frequent decisions.

10.2 METSM as a non-learning counterpoint

Jiang et al. [20] formulate METSM for edge heterogeneous multiprocessor scheduling with makespan and total energy as objectives. Its decision variables include execution order, processor assignment, and DVFS level. An iterated greedy-based multiobjective optimizer uses problem-specific reconstruction and local search. This is important prior art because it already integrates several variables that a new DRL proposal might otherwise describe as novel. The relevant comparison is between alternative mechanisms for solving a coupled problem, not between an integrated learner and a deliberately incomplete heuristic.

A multiobjective optimizer can produce several trade-off solutions instead of hiding preferences inside one weighted reward. This helps distinguish changes in solution quality from changes in the chosen energy–latency balance. The limitation for online deployment is that search time must fit the decision budget. A learned policy may amortize optimization into training, but that advantage should be evaluated against the actual workload horizon and the cost of obtaining training experience.

10.3 A fair comparison between search and learning

The two approaches answer related but different questions. Graph-based DRL seeks a reusable mapping from observations to decisions; METSM searches for high-quality schedules for the specified problem instance. A fair experiment should equalize available workload information and clearly separate offline planning from online reaction. Small instances can support an exact reference solution, while larger cases can test scalability and responsiveness. The strongest claim would show where learning improves the achievable energy–service trade-off under a fixed runtime budget. Merely outperforming a makespan-oriented list scheduler would not establish superiority over energy-aware multiobjective scheduling.

11. Multi-Task, Multi-Deadline DVFS: 2024–2025

11.1 Publication status and problem expansion

Xinyi Li, Zhou, Wang, and Lin [8] extend temporal DRL-based DVFS to workloads with multiple tasks and deadlines. The work first appeared as an arXiv manuscript in 2024 and has a verified journal publication in IEEE Transactions on Sustainable Computing in 2025. This expanded review cites that publication while identifying the accessible May 2025 manuscript as the basis for the implementation details discussed below. This distinction avoids treating a now-published study as merely an unverified preprint.

The multi-task problem is more complex than controlling a single job against one deadline. A frequency reduction can be harmless to one task but delay another task that shares the processor. Tasks can overlap in time, and the system's aggregate utilization does not reveal which deadline is most urgent. The manuscript uses Double DQN and alternative temporal encodings to represent the evolving workload and derive frequency decisions. This supports a progression toward more realistic timing requirements while retaining a relatively lightweight observation design.

11.2 Evidence and experimental boundary

The inspected manuscript evaluates fixed workloads containing three, five, and eight tasks, together with randomly generated task sets, on a Jetson Nano 2 GB. It assumes independent tasks with predefined start times and restricts the reported hardware action choices to two frequency/voltage levels. These conditions matter because they bound both the scheduling uncertainty and the action-space size. A method that works well with two operating choices is not automatically equally efficient with many clusters and operating points.

The abstract reports 3%–10% power savings against built-in Linux governors. The metric should be preserved when quoting the result. Equal percentage energy savings would require compatible execution durations and matched work, which cannot simply be assumed. The study also does not establish arbitrary DAG scheduling or task placement across heterogeneous CPU types. Its contribution is a multi-task deadline-aware governor, not a complete replacement for every component of a heterogeneous operating-system scheduler.

11.3 What a follow-up study should isolate

The supplied project [25] does not yet have identifiable tasks or deadlines. It should add those semantics before borrowing the multi-deadline claims of this study. It should compare frequency-only control with joint placement–frequency control, rather than assuming that adding placement necessarily helps. Additional counters should be introduced only when they resolve an observed ambiguity that temporal utilization cannot capture. Testing should also distinguish adaptation within a repeated task set from generalization to new task compositions. These experiments would reveal whether the limiting factor is representation, action granularity, hardware coupling, or distribution shift. That diagnosis is more scientifically informative than reporting one larger percentage improvement under a different benchmark.

12. FiDRL and the Economics of Controller Invocation

12.1 From choosing actions to choosing decision times

Jingjin Li et al. [9] introduce FiDRL, which incorporates the agent's invocation interval into its decision process. The motivating observation is that a DRL controller consumes computation and energy whenever it runs. Reducing model size through pruning or quantization addresses the cost per decision, but not the number of decisions. FiDRL addresses this temporal dimension and applies its approach to inter-task and intra-task DVFS. For a small edge device, this is a major conceptual improvement over evaluating the scheduler as if inference were free.

A controller should act when new information can change a consequential decision. If the workload remains stable, frequent invocations may provide little value. If a critical task arrives or a thermal limit approaches, waiting too long can be harmful. Thus, invocation frequency creates its own energy–latency trade-off. This does not imply that a learned interval is always preferable: a well-chosen event-triggered heuristic may be competitive and should be included as a baseline.

12.2 A useful break-even model

Let the incremental cost of one observation-and-decision cycle be c joules and let the controller run n times during an evaluation horizon. Its direct control cost is approximately n times c, before considering changes in cache behavior or other indirect effects. If its scheduling choices save less than that amount relative to a matched baseline, the device-level benefit is negative. This is an explanatory accounting model proposed here, not a performance result reported by FiDRL. The purpose is to make the overhead budget visible before selecting a large policy.

Training introduces another timescale. A policy may save energy during inference yet require substantial data collection or online updates. A deployment break-even estimate divides one-time training energy by average positive net power saved, under a representative operating regime. The estimate becomes unreliable if workload drift frequently forces retraining. Accordingly, useful reporting includes both inference-only savings and a lifecycle view that states which training costs are included and how frequently they recur.

12.3 Implications for architecture and experimentation

FiDRL supports separating a fast actuation path from more expensive learning and monitoring activities. It also motivates ablations that hold policy architecture constant while changing invocation rules. Otherwise, gains from fewer decisions can be confused with gains from better decisions. Decision intervals should be evaluated under normal load, burst arrivals, and thermal disturbances. The relevant outcome is not simply lower invocation cost but preserved service quality with lower total energy. A controller that saves energy by missing urgent changes has optimized its own activity at the expense of the application contract.

13. Integrated Heuristics and Complex Learning Frameworks

13.1 Predictive scheduling without DRL

Liu et al. [10] integrate task prioritization, core-aware mapping, and predictive DVFS for heterogeneous multicore edge computing. The article reports a 20.9% energy reduction with a 2.4% deadline-miss rate under its evaluation. These are author-reported results, not an expected improvement for every device. The nonzero miss rate also makes clear that the finding is not a hard-real-time guarantee. Its principal role in this review is to establish a serious integrated non-learning baseline: mapping and frequency decisions can be coordinated without a neural policy.

Kazemi et al. [11] combine genetic and ant-colony optimization with slack-based DVFS for time-constrained DAG applications in heterogeneous edge environments. The method illustrates a different strategy: construct a schedule, then exploit available slack to reduce energy. Its relevance to local CPU scheduling depends on the time available for optimization and the accuracy of task-cost estimates. An optimization method designed for a broader edge environment may not fit a short kernel scheduling interval, but it can still serve as an offline or slower-timescale comparator.

13.2 FR-EAHTS and the burden of architectural complexity

Farooq, Zafar, and Samad [12] present FR-EAHTS, combining transformer-based representation, adaptive PPO, federated learning, hierarchical load balancing, and DVFS. The accessible publication record establishes this proposed integration, but does not by itself resolve whether every communication, monitoring, training, and inference cost is included within a comparable device-energy boundary. Consequently, the framework is best treated here as evidence of a broad design direction, not proof that its full complexity is necessary on a small edge board.

When several components are introduced together, the explanation of improvement becomes ambiguous. A better result could arise from additional workload information, revised priorities, a more favorable frequency configuration, or the learning mechanism. Ablations should remove components one at a time while preserving training budget and service requirements. Federated learning also has a different motivation from single-device control: sharing experience across devices may help adaptation, but communication and heterogeneous hardware can complicate aggregation. Those costs and assumptions need explicit treatment.

13.3 A baseline-selection principle

The appropriate comparison is not “AI versus traditional scheduling” in the abstract. It is a matched comparison among implementations with the same legal actions, workload information, and service obligations. Predictive heuristics, metaheuristic planning, small learned policies, and complex federated controllers occupy different points in the cost–adaptability space. The proposed research should identify the operating regime in which a particular controller is worthwhile. An improvement obtained only when simpler methods receive less information is not compelling evidence that the learning architecture itself is responsible.

14. Joint Heterogeneous Control on Real Edge Hardware

14.1 Haoyu Wang's 2025 thesis as close prior art

Haoyu Wang's master's thesis [13] directly addresses joint DVFS and task scheduling on a heterogeneous big.LITTLE edge platform. The controller selects both a CPU cluster and an operating performance point on an ODROID-N2+, conditioned on a soft target execution time. Its implementation separates a lightweight executor from an on-device Double DQN learner. This is particularly close to the proposed topic because it moves beyond frequency-only control and demonstrates a concrete path from a learned policy to heterogeneous CPU actuation.

The thesis reports per-cycle energy reductions of up to 10.6% for keyword spotting and 7.3% for a YOLO-lite workload relative to Ondemand under its evaluated timing conditions. These findings should retain their comparison context and thesis status. They are not interchangeable with journal results from other boards or with average savings over arbitrary tasks. Nevertheless, their existence is important: it would be inaccurate to claim that learned cluster selection and DVFS have never been jointly deployed on heterogeneous edge hardware.

14.2 Why the task model matters

The principal scope is single-task, single-threaded, periodic execution under soft timing targets, with a primary emphasis on CPU-bound workloads. This controlled setting allows the learner to associate progress, operating-point residency, and timing requirements with energy outcomes. It does not establish a general scheduler for mutually interfering tasks, dependency graphs, or mixed-criticality execution. Memory-bound behavior is especially relevant because changes in CPU frequency may produce little change in useful throughput, weakening utilization-based interpretations of remaining work.

The target-execution-time conditioning also separates response adaptation from retraining. A policy can respond to a different timing target through its input representation without changing its parameters. That is valuable, but it should not be confused with strong transfer across application families. A new workload may have a different relationship among frequency, migration, and execution progress even when its deadline is numerically similar. Generalization must therefore be measured with task composition and resource behavior changed, not just timing targets.

14.3 A narrower and more defensible research opportunity

The natural next question is whether joint control remains beneficial under multiple active tasks, shared resource contention, and realistic workload shifts while preserving fairness. This extends a demonstrated implementation rather than claiming the first combination of existing ideas. A comparative study should reproduce a compact joint controller where possible, then add one capability at a time: multiple queues, deadline-specific features, contention indicators, or selective adaptation. Positive results would show which extension produces a reproducible net benefit. Negative results would also be valuable if they identify the conditions under which a simpler controller is preferable.

15. Partial Observability Becomes Explicit in 2026

15.1 Hussain and Lin's reformulation

Hussain and Lin [14] examine learned DVFS optimization on a resource-constrained Jetson Nano and explicitly formulate the control problem as partially observable. Their approach incorporates temporal dependencies across observations and modifies reward and exploration behavior. The study reports performance improvement of up to 17% compared with the evaluated prior solution, with a small increase in power. This is not a 17% energy-saving result. Preserving that distinction is necessary because performance, power, and total energy can move differently.

The conceptual contribution is recognizing that commonly available telemetry is not a complete state description. Utilization, temperature, and frequency do not uniquely identify an application phase, future arrivals, memory behavior, or remaining work. Two apparently identical observations can require different actions because they follow different histories. A partially observable formulation makes this ambiguity explicit, rather than assuming that a few counters satisfy the Markov property merely because they are fed to a neural network.

15.2 Choosing an affordable memory representation

The repository [25] uses a three-value instantaneous observation. A history window, temporal summary, or recurrent hidden state is an extension to evaluate, not an existing feature. A longer history may capture more context but increase computation, memory, and training difficulty. The best choice depends on the temporal scale of relevant events. A small periodic application may be adequately described by elapsed progress and recent utilization, whereas overlapping tasks can require explicit queue and deadline information. No representation should be declared sufficient without an experiment showing which ambiguities it resolves.

The inspected study evaluates two application tasks, so its findings do not establish general multi-task heterogeneous CPU scheduling. However, it gives a clear methodological direction for such work: test whether historical information improves the decisions that matter, not merely the prediction accuracy of an auxiliary model. For placement, this could mean distinguishing a genuinely compute-intensive phase from one stalled by memory contention. For DVFS, it could mean recognizing whether a utilization burst is likely to persist until a deadline.

15.3 Robustness under observation imperfections

Real deployment adds missing samples, delayed telemetry, sensor noise, and discrepancies between requested and effective frequency. These are natural stress tests for a partially observed controller. The evaluation should vary observation delay and noise while keeping task traces fixed, then report energy and service degradation. A policy that requires perfectly aligned measurements may perform well in a laboratory trace but be difficult to maintain in an operating system. The broader lesson of [14] is therefore not simply to add a recurrent layer: it is to make information limitations part of the problem definition and to test the consequences explicitly.

16. HiDVFS: Hierarchy, Thermal Feedback, and Runtime Screening

16.1 A coordinated multi-agent design

The July 2026 revision of HiDVFS [15] proposes hierarchical multi-agent control for real-time OpenMP DAG workloads. Separate agents address core-count and frequency selection, thermal grouping, and task priorities. The manuscript includes feasibility gating and a calibrated response-time shield, and reports evaluation on Jetson TX2, Orin NX, and RubikPi. In its cross-platform deadline-aware comparison, it reports 15%–18% energy reduction relative to fixed maximum frequency. The inspected record identifies the work as a preprint under review, not a confirmed journal publication.

The hierarchy reflects a sensible decomposition of coupled decisions. Frequency, available parallelism, temperature, and task priority evolve at different rates and have different observable effects. Yet decomposition also introduces coordination questions: one agent's action changes the environment encountered by another. A hierarchical method should therefore be compared not only with an unstructured large agent, but also with a compact joint policy and a heuristic hierarchy. Otherwise, benefits from separating timescales can be mistaken for benefits unique to multi-agent learning.

16.2 Interpreting the objective and constraints correctly

HiDVFS uses a makespan-focused learning objective with energy and thermal considerations. It should not be described as an unqualified energy-optimal scheduler. The manuscript's deadlines are soft, and its calibrated prediction bounds are empirical rather than worst-case timing guarantees. A response-time shield can reject actions whose predicted upper bounds exceed a deadline, but prediction reliability depends on calibration assumptions and the relationship between deployment and calibration data. Distribution shift can weaken that relationship.

The reported control cycle is approximately 2 ms and is discussed as appropriate for second-scale jobs rather than microsecond-level control. This limitation is valuable information, not an incidental implementation detail. It places the method in an application-level or coarser-grained scheduling regime. A new project targeting much shorter jobs would need to demonstrate a smaller observation and inference path rather than borrowing the same architecture and assuming feasibility.

16.3 How this changes the research gap

By 2026, hierarchy, thermal state, DAG workloads, runtime screening, and cross-platform evaluation are no longer unexplored combinations. A defensible follow-up should identify a narrower weakness: energy-first optimization at equal service, robustness to unseen task interference, decision overhead on smaller processors, or constraints appropriate to a specific mixed-criticality deployment. Reproduction should preserve native baseline objectives or explain any changes when adapting methods to new workloads. This is particularly important when a policy's reward favors makespan while its comparator was designed for a different energy–performance balance. Objective alignment is necessary before interpreting relative gains as algorithmic superiority.

17. Broader 2026 Evidence and Cross-Domain Transfer

17.1 Joint chip choice, offloading, and DVFS

Shao et al. [16] present a multi-objective scheduling framework for heterogeneous AI chips using hybrid-action PPO. It selects offloading destinations, physical chips, and DVFS settings using workload, network, thermal, and timing information. The paper reports simulation and hardware-in-the-loop evaluation. This is useful evidence for coordinated control and feasibility-oriented action masking, particularly when discrete placement decisions interact with bounded frequency adjustments.

Its scope, however, includes CPU, GPU, NPU, and FPGA resources across edge-cloud execution. Improvements can arise from accelerator selection or avoidance of unnecessary communication, not just better local CPU placement. The study should therefore inform architecture design without being used as proof of the same gain on a single big.LITTLE CPU. A proposed local scheduler would need to remove or hold constant offloading and accelerator choices before attributing an improvement to CPU scheduling alone.

17.2 What recent surveys contribute

Hossain, Banu, and Arefin [17] provide a 2026 synthesis of CPU scheduling across classical, real-time, multicore, production operating-system, and learning-based paradigms. The accessible abstract emphasizes trade-offs and the lack of a universally dominant policy, along with reproducibility and certifiable learning. Although published in 2026, its literature coverage extends through 2025. It is therefore a contemporary synthesis rather than a substitute for examining the new 2026 primary studies individually.

Hou, Agos Jawaddi, and Ismail [24] review DRL-based energy-efficient scheduling in cloud environments, comparing energy models, state and action definitions, reward construction, and evaluation settings. Its contribution is a useful methodological taxonomy. Its cloud scope and earlier review window mean it cannot establish what is currently best for embedded CPU scheduling. Together, the two surveys support transparent model and evaluation descriptions while reinforcing the need to avoid transferring conclusions across architectural boundaries.

17.3 A disciplined transfer principle

Cross-domain transfer is most credible at the level of mechanisms: representing dependencies, decomposing actions, encoding history, or constraining exploration. It is weaker at the level of reported percentages and least credible when hardware capabilities differ. A method should be reformulated for the local action space and service contract before its results are compared. The review’s central thesis follows: the field has many plausible components, but selecting and validating the smallest effective combination remains a substantial systems problem. Architectural complexity is justified only when its incremental benefit survives matched workload, information, and overhead controls. This principle guides the comparison matrix and research agenda that follow.

18. Repository Architecture and Evidence Map

The implementation reviewed is AdityaRezan/energy-aware-drl-scheduler at commit 2093bec5206c051cd4d806b2f1bd8a58634e42e1, inspected on 16 October 2026 [25]. The table separates what each artifact contains from what its presence alone establishes.

Artifact	Observed implementation	Evidence boundary
edge_env.py	Gymnasium model; three normalized observations; four coefficient-based actions.	Aggregate backlog and thermal simulation; no task-level or real DVFS control.
ppo_edge_scheduler.zip	Saved actor–critic archive and training configuration metadata.	Not loaded or evaluated by the current Streamlit/C path.
main.c	3→64→64→4 ReLU inference and a synthetic paired comparison.	No CPU affinity, CPUFreq, energy sensors, or actual Linux governor experiment.
weights.h	Dummy arrays explicitly supplied to test compilation.	Current C action is always zero; not a demonstrated export of the PPO actor.
app.py; Makefile	Dashboard compiles/runs C; command-line scale, queue, and horizon controls.	Hardware presets alter a power scale, not calibrated architecture parameters.
benchmark_results.png	Static power, queue, and thermal plots.	Generation script and evaluation traces are absent from the inspected tree.
README; requirements	Project description and unpinned package names.	Demo URL is a placeholder; training/export/evaluation provenance is incomplete.

18.1 Saved-model evidence

Plain-text metadata in ppo_edge_scheduler.zip identifies an ActorCriticPolicy with three observations and four discrete actions. It records a learning rate of 0.001, gamma 0.99, 2,048 rollout steps, batch size 64, ten update epochs, and 30,720 collected timesteps against a requested total of 30,000. It also records seed=None and Stable-Baselines3 2.9.0. These are saved configuration fields, not independently reproduced training results. The serialized policy was not executed during this review [25].

18.2 Relationship to the literature

Temporal governors [6], [8] show why history can matter; the current observation is only an instantaneous three-value vector. Joint heterogeneous control [13] motivates real cluster and operating-point actuation, absent from this simulator. Invocation-aware work [9] motivates measuring controller cost, not currently included. The repository thus provides a starting environment and interface, while the literature identifies specific next steps rather than supplying evidence that those capabilities already exist.

19. Implemented State, Actions, Reward, and Dynamics

19.1 Observable state and abstract configurations

The Python state is s = (q, θ, c): normalized queue load q, normalized thermal state θ, and the binary core-class flag c. reset initializes θ to 0.3, c to zero, and q from a uniform distribution between 0.1 and 0.9. Every observation coordinate is bounded between zero and one. The four actions use the coefficients below; they are not specified physical frequencies, voltages, core counts, or measured capacities [25, edge_env.py:10–37].

Action	Python power P	Capacity C	Temp. increment T	Core flag
0	0.2	0.2	0.01	0
1	0.5	0.4	0.02	0
2	1.2	0.7	0.05	1
3	2.5	1.0	0.10	1

19.2 Transition and reward equations

For arrival A sampled uniformly from 0.1 to 0.4, action capacity C_a, and thermal increment T_a, the implemented transitions are:

$$q_{next} = \mathrm{clip}(q + A - C_a, 0, 1)$$

$$\theta_{next} = \mathrm{clip}(\theta + T_a - 0.02, 0, 1); \; c_{next} = 1 \text{ if } a \geq 2, \text{ else } 0.$$

$$\text{reward} = -[0.4\, P_a + 1.5\, q_{next} + 2\, I(\theta_{next} > 0.8)].$$

The queue term is a backlog proxy, not a measured waiting time. No job release, completion timestamp, deadline, or per-task execution record is maintained. The thermal term is a reward penalty, not an actuator or capacity-throttling mechanism. The C routine approximates the same state transitions with a scaled power table but does not calculate this reward. Future documentation should state the time and work units before translating these quantities into latency, throughput, or energy claims.

19.3 Training and reproducibility implications

The Python step method always returns terminated=False and truncated=False. This is a continuing environment, so evaluation needs an explicit external horizon; it does not justify invented per-episode returns. Although reset calls the Gymnasium superclass with a seed, subsequent draws use global numpy.random rather than the environment generator self.np_random. Reset-seed control alone therefore does not establish reproducible arrivals. The C program uses srand(42), but its random stream is not numerically equivalent to NumPy's [25].

These details define the current research problem more accurately than a generic constrained scheduler formulation. The observation has no task graph, deadline slack, domain frequency, historical encoding, or peak-power parameter. Such features are possible extensions motivated by [7], [13]–[15], not implemented contributions. A sound first experiment should train and compare policies on this stated aggregate model before expanding its scope. Conservation of work and stronger baselines are higher-priority corrections than increasing network complexity.

20. Local C Checks and Interpretation of Reported Savings

20.1 The currently supplied weights select action zero

weights.h explicitly describes its arrays as dummy weights for testing the compiler pipeline. C aggregate initialization fills unspecified elements with zero. Only the first path through the three-layer calculation has positive weights, and all biases are zero. For nonnegative inputs, action zero has a nonnegative logit while the other logits are zero; the strict greater-than comparison also retains action zero when all logits tie. The current C actor therefore selects action zero throughout the declared observation domain [25, weights.h:4–10; main.c:22–57].

A review-only local harness tested 1,331 evenly spaced points in the cube from zero to one and observed action zero at every point. This corroborates the algebraic finding; it is not a PPO evaluation. The original C program was also compiled and run with peak scale 15, initial queue 0.5, and 100 steps. The following results use its fixed C seed and separate bookkeeping instrumentation; served and discarded quantities are normalized work units, not counts of physical jobs.

Local check: 15-scale, 100 steps	Current C actor	Always action 3
Actions selected	0 at all 100 steps	3 at all 100 steps
Accumulated modeled power	120.00	1500.00
Served normalized work	20.0000	26.3059
Work discarded by clipping	5.3059	0.0000
Final normalized backlog	1.0000	0.0000

20.2 Why 92% is not validated PPO energy improvement

The printed improvement follows directly from $1 - 0.08/1.00 = 0.92$: the current actor always chooses the lowest power ratio, while the baseline always chooses the highest. It is independent of learned PPO decisions. In the local trace, the candidate also serves less work, reaches a full queue, and silently loses excess demand through clipping. The comparison therefore does not preserve service equivalence. The program's "Joules" labels additionally lack an explicit step-duration definition, as explained in Section 3.

20.3 Static plot versus current executable

benchmark_results.png depicts varying low-power decisions and a relatively small queue. The current placeholder-weight C program cannot generate that varying action behavior. This does not establish that the image is invalid; it may describe a different evaluation path or earlier run. However, the snapshot provides neither a figure-generation script nor trace data linking the image to the saved model and current sources. It should be treated as an unverified repository illustration, not reproduced evidence for the present dashboard. No physical energy savings, deadline guarantees, or trained-policy C equivalence are established by these checks [25].

21. Repository-Aligned Architecture and Revision Priorities

21.1 Preserve and connect the existing components

The repository already provides a compact Gymnasium model, a PPO artifact, a native simulator, a build path, and an interactive dashboard [25]. The priority is to connect these components transparently, not to replace them with a larger architecture. Add a versioned training entry point that records source revision, environment parameters, seed, package versions, training budget, and checkpoint hash. The current requirements.txt lists packages without versions; archive metadata can inform a tested environment file but does not prove compatibility with every package release.

Next, replace dummy C parameters through a documented exporter for the exact actor used by the saved PPO policy. Export ordering, weight layout, biases, activation functions, preprocessing, and deterministic action selection must match. The C code explicitly uses ReLU; the snapshot does not establish that this equals the saved actor's activation. No equivalence claim should be made from matching layer dimensions alone. Compare logits and actions over boundary states, a fixed grid, and held-out trajectories before calling the native path an exported PPO scheduler.

21.2 Correct the simulator and benchmark contract

Use one shared configuration specification for the Python and C action tables, arrivals, cooling, and time units. Replay a common arrival trace rather than expecting different random generators to produce identical samples from the same seed. Replace silent backlog clipping with explicit capacity, rejection, or queue-overflow accounting. Track offered demand, served demand, residual work, and dropped work. Define energy as modeled watts multiplied by an explicit duration, with labels that distinguish simulation from measurement [25].

Dashboard labels should identify the current comparator as a synthetic always-maximum policy. Hardware presets should be described as power-scale presets until board-specific capacity and thermal parameters are calibrated. Remove or replace the placeholder live-demo URL in README.md only when a real deployment address exists. Extend compilation-error handling to runtime exit codes and complete result-parsing checks so failed simulations do not produce misleading output. These are implementation recommendations, not changes already committed to the project.

21.3 Add capability in an evidence-driven order

After actor equivalence and accounting are established, evaluate history features [6], [14], invocation cost [9], and contention-aware observations [19] through ablation. Task deadlines require identifiable jobs, not merely a larger queue penalty. Real heterogeneous placement requires a board topology and legal controls, as in [13]. Action masks can enforce valid interfaces, but formal safety needs justified assumptions [23]. The correct sequence is reproducibility, semantic consistency, service-equivalent evaluation, then hardware extension. Each stage should produce its own evidence before the document advances the corresponding claim.

22. Evaluation Plan for the Supplied Repository

22.1 Establish invariants before reporting improvements

Unit tests should check reset behavior, action validity, observation bounds, thermal updates, external evaluation horizon, and conservation of work. Fix random-number handling before treating reset(seed) as a reproducibility control. Test Python and C transitions against the same explicit arrival sequence, including empty and saturated queues. The model archive should load in a pinned environment, and its actor output should be compared with the exported C implementation. The present review does not claim these tests already pass [25].

A conservation assertion should relate initial backlog plus arrivals to served work plus final backlog plus explicitly rejected work. That assertion would expose the current clipping loss measured in Section 20. A second assertion should check the dimensional energy calculation against a known constant-power action and timestep. A third should confirm that changing a hardware profile affects the intended physical parameters, not merely a numerical multiplier used for every policy.

22.2 Use informative policy baselines

Evaluate all four constant actions, a queue-threshold heuristic, a thermal-aware heuristic, the saved PPO policy, and its verified C export. Constant action one is particularly important: its capacity is 0.4, the maximum arrival in the current environment, and its thermal increment equals the fixed cooling term. Under those assumptions, its queue cannot increase from arrivals alone and its thermal state remains unchanged. It is therefore a more informative low-complexity comparator than action three alone. This observation follows from the source model, not from a claim of physical hardware optimality [25].

Run matched arrival traces over multiple seeds and workload regimes. Include heavier arrivals or burst processes only as separately specified scenarios; they are not part of the current uniform-arrival model. A policy trained on the original regime should be tested unchanged before any fine-tuning result is reported. Training and test traces should be separate, and parameter tuning should not use the final comparison traces.

22.3 Metrics and a path to hardware evidence

For the aggregate simulator, report accumulated modeled energy with timestep assumptions, served work, rejected work, final and mean backlog, thermal-threshold exposure, and action frequencies. Do not report actual latency or deadline misses until jobs and timestamps are implemented. Report confidence intervals and per-trace results, not just one percentage. Reproduce each figure from saved data and a versioned script.

For a later board experiment, add CPU topology, operating-point tables, measured power, actuation latency, controller overhead, and actual completion timestamps. Compare against real governors only when the experiment controls and records them. The deployment and evaluation lessons of [2], [5], [9], [13], and [19] then become directly applicable. Until that stage, describe conclusions as simulator findings rather than evidence of ARM hardware energy savings.

23. Synthesis, Claim Boundaries, and Project Deliverables

23.1 What the repository currently contributes

The supplied project brings together a simple energy-aware control environment, a saved PPO model artifact, a C inference scaffold, and a Streamlit demonstration [25]. That is a coherent prototype foundation. The environment makes a power–backlog–thermal trade-off explicit, and the C comparison uses a repeated seed to pair arrival streams within its own simulation. These are useful starting design choices. They should be documented accurately without implying that the current native path already runs the learned actor or manages actual heterogeneous CPU resources.

The inspection also identifies concrete boundaries. The queue is aggregate rather than task-level; the thermal state is normalized and uncalibrated; operating points are coefficient tables rather than voltage/frequency pairs; the dashboard invokes a simulator rather than an OS interface; and the supplied C parameters are placeholders. Its 92% displayed reduction arises from fixed action coefficients and is not a reproduced PPO energy-saving result. The local bookkeeping check additionally shows unequal serviced work and discarded backlog, preventing a service-equivalent interpretation.

23.2 How the literature supports the next contribution

Temporal and partially observed studies [6], [8], [14] motivate richer information only after the basic model is reproducible. Fairness and contention research [2], [19] motivates explicit multi-task service rather than a single queue penalty. FiDRL [9] motivates counting invocation costs. Joint heterogeneous control [13] and HiDVFS [15] show that placement, frequency control, and runtime screening already have close prior art. Consequently, a defensible contribution would demonstrate a verified PPO-to-C pipeline and honest, service-matched simulator evaluation, followed by a bounded hardware extension where feasible.

Such a contribution need not claim universal scheduling superiority. It can establish that a particular lightweight policy offers a reproducible trade-off under stated workload, observation, and action assumptions. It can also identify when a constant or heuristic policy is sufficient. Negative results are useful when they reveal that a model is too simple, an apparent gain comes from baseline choice, or an export changes policy behavior.

23.3 Recommended deliverables and limitations

The next repository release should include version-pinned training, checkpoint provenance, an actor exporter, Python–C equivalence tests, common arrival traces, conservation tests, baseline implementations, raw evaluation data, and plot-generation scripts. A deployment claim additionally needs board calibration and measured overhead. These are recommended changes; the repository itself was not modified by this review.

This document remains a narrative literature review combined with a snapshot-specific implementation assessment. Published findings were not reproduced, the saved PPO policy was not executed, and the local C checks are simulator diagnostics rather than hardware experiments. The project reference [25] identifies the repository and commit so that later revisions can be assessed against the same evidence standard. The central standard is demonstrable control quality under explicit assumptions, rather than an impressive percentage obtained from incomparable service or unverified model integration.

24. Extended Review Methodology and Evidence Grading

24.1 Purpose of this supplement

The preceding twenty-three sections were written as a narrative review tied to a single pinned repository snapshot. A narrative is good at explaining why a body of work matters, but it is weaker at showing how the evidence was collected and how much weight each individual claim can carry. This supplement closes that gap. It records the search boundary, the coding frame used to classify the twenty-five references, the grading rubric applied to every quantitative claim, and the checks that were actually run against the code. No new source is introduced here. Every statement is grounded in references [1] to [25] or in a reproducible command in the project repository.

24.2 Search boundary and selection protocol

The literature window opens in January 2020 and closes on 15 October 2026. Three index families were searched: IEEE Xplore and the ACM Digital Library for conference and journal records; ScienceDirect, SpringerLink and Wiley for journal records; and arXiv, Google Scholar and institutional repositories for preprints and theses. Queries combined a scheduling term, an energy term and a learning term, for instance energy-aware DVFS with deep reinforcement learning at the edge, heterogeneous multicore task scheduling with reinforcement learning, and thermal-aware CPU governors framed as partially observable control. The reference lists of the three reviews in the set ([17], [23], [24]) were also scanned backwards to recover older but load-bearing work such as [18] and [21].

Inclusion required a dated record, an identifiable evaluation substrate, and a stated energy or thermal objective. Exclusion removed records whose energy model was purely notational, records concerned only with radio or network resource allocation, and records whose sole contribution was a survey of other surveys. After de-duplication, 61 candidates were screened, 39 were read in full, and 25 were retained: 22 journal or conference papers, one master's thesis, one preprint, and the software artefact itself ([25]). The selection is deliberately focused rather than exhaustive, as Section 1.3 already notes.

Publication status was checked one record at a time. Reference [8] was confirmed as a journal article with an accessible preprint; [16] is a preprint explicitly marked as under review; [14] is a thesis rather than a peer-reviewed paper. That distinction matters. A 2026 result resting on a preprint carries different evidential weight from a 2020 result with five years of independent follow-up. Roles for all twenty-five entries appear in the annotated bibliography on pages 36 to 38.

24.3 Evidence grading rubric

Each quantitative claim extracted from the literature was assigned one of four grades. Grade A denotes a measurement on physical hardware with a stated baseline, a stated workload, and either a reported variance or repeated trials. Grade B denotes a result from a cycle-accurate or otherwise validated architectural simulator with documented parameters. Grade C denotes an abstract numerical simulation whose power model is analytic rather than calibrated. Grade D denotes an illustrative example, a figure without a reproducible configuration, or a claim quoted from another source without independent verification.

Grade	Substrate	What the number may be used for	Typical example in this set
A	Physical board	Direct comparison against a named baseline on stated hardware	Measured DVFS traces in [6], [15]
B	Validated simulator	Trend and sensitivity analysis, not absolute savings	SoC-level studies in [18], [20]
C	Analytic model	Mechanism and ablation studies only	Aggregate queue models in [3], [25]
D	Illustrative figure	Motivation and hypothesis generation	Concept diagrams in surveys [17], [24]

The grade is not a judgement about the authors. A careful simulation graded B may be more useful to a student project than a hardware measurement graded A whose board revision, compiler flags and workload are unavailable. It is a statement about how far a number can travel. Grade C and D figures should not be quoted as expected savings on a real device, and they must never be compared directly with measurements taken on a different substrate.

24.4 Extraction fields

For each retained record a single reviewer extracted twelve fields into a flat table; a second reviewer checked the entries against the source. Disagreements were resolved by returning to the paper rather than by splitting the difference. The fields were chosen so that every claim in Sections 5 to 17 can be traced back to a specific attribute of a specific paper.

#	Field	Why it was recorded
1	Year and venue	Positions the work in the 2020-2026 timeline
2	Publication status	Separates journal, conference, thesis and preprint
3	Evaluation substrate	Determines the evidence grade A to D
4	State representation	Shows whether history or partial observability is used
5	Action space	Reveals whether DVFS and placement are jointly controlled
6	Reward or objective	Identifies the energy, thermal and deadline terms
7	Learning algorithm	PPO, DQN, actor-critic, or non-learning heuristic
8	Baselines	Indicates whether comparisons are like-for-like
9	Reported saving and units	Records what was actually measured
10	Variance information	Whether seeds, error bars or trials are given
11	Controller overhead	Invocation interval and inference cost
12	Artifact availability	Code, traces, or neither

24.5 Threats to validity

Construct validity is the most serious concern in this topic. The word energy is used in the literature for at least four different quantities: instantaneous power drawn from a rail, energy integrated over a task, energy per completed unit of work, and a dimensionless power ratio chosen by the author. Several papers report the first two without normalising by work, which makes a slow policy look efficient. Sections 3 and 20 of this review separate these meanings explicitly, and the remediation in Section 26 enforces the normalised form in code.

Internal validity is threatened by unequal seeds, unequal episode lengths, and silently clipped queues. A learned controller and a baseline must see the same arrival stream; otherwise the comparison measures the random number generator. External validity is limited because most studies use a board generation that is already obsolete, or a simulator calibrated to one. Conclusion validity is limited by the small number of independent runs in several papers, and by the common practice of reporting only the best of several training seeds.

24.6 How the repository was inspected

Static inspection covered the README, the Gymnasium environment, the Streamlit application, the C sources and headers, the exported weight header, the Makefile, the dependency list, the saved-model archive metadata and the benchmark image. Dynamic inspection compiled the C program with GCC, ran it under a fixed random seed, swept a grid of observations through the inference function, and parsed the printed accounting. All checks ran in a sandbox on a Linux host; no board was connected and no operating-system setting was changed.

Only plain-text metadata was read from the saved PPO archives. Their serialized Python objects were not loaded for execution, so the review makes no claim that the stored policy was retrained or shown equivalent to the C actor. That chain of evidence is now testable in the revised repository through the export script and the parity test described in Section 26. The inspection boundary remains the commit recorded in Section 1.2, which is preserved in the history of the current repository.

Reading note. Sections 24 and 25 are methodological. Readers interested only in the corrected measurements should go directly to Section 26 on page 30.

25. Comparative Evidence Synthesis Across the Twenty-Five Sources

25.1 A five-axis coding frame

Reading twenty-five papers in chronological order makes the field look like steady progress. Reading them by design axis makes the real pattern visible: the community has been solving five loosely coupled problems at once, and very few papers address more than two. The five axes are temporal representation, service and contention modelling, action-space structure, invocation cost, and evaluation realism. Each retained reference was coded as primary, secondary or absent on each axis. The resulting map is summarised in the table below and discussed in the rest of this section.

Temporal representation separates work that feeds a single instantaneous observation to the policy from work that encodes a window, a phase label, or an explicit belief state. Service and contention modelling separates aggregate queue abstractions from per-task deadline and shared-resource models. Action-space structure covers whether frequency, core class and placement are chosen independently or jointly. Invocation cost concerns how often the controller runs and what it costs to run it. Evaluation realism is the evidence grade of Section 24.3.

25.2 Evidence map

Period	References	Dominant axis	Typical substrate	Modal grade
2020-2021	[1], [18], [21]	Thermal control, simulation infrastructure, action compression	Hardware testbed or validated SoC simulator	A to B
2022	[2], [3], [4], [5], [19], [23]	Fairness, contention, workload phase, transfer limits	Hardware counters and simulation	A to B
2023	[6], [22]	Temporal encoding, deadline-aware offloading	Embedded deployment	A to B
2024	[7], [20], [24]	DAG structure, multiobjective heuristics, taxonomy	Simulation	B to C
2025	[8], [9], [10], [11], [12], [13]	Invocation economics, multi-deadline DVFS, integrated frameworks	Mixed simulation and testbed	B to C
2026	[14], [15], [16], [17]	Partial observability, hierarchical control, cross-domain transfer	Testbed and survey	B to C
Artefact	[25]	None: prototype with analytic power ratios	Abstract numerical simulation	C

Two features of the map deserve attention. First, the highest evidence grades cluster in the earliest years, when the problems were narrower and the hardware was available. The more ambitious 2025-2026 studies tend to be graded B or C because their integrated claims are harder to validate end to end. Second, the artefact under study sits alone at grade C. That is not a defect in the student project; it is an accurate description of what a software-only prototype can establish.

25.3 Where the evidence converges

Across the set there is strong agreement on four points. Energy and latency cannot be optimised independently, so any credible objective must contain both a work term and a power term. Fixed governors are a weak baseline once workloads become phase-dependent. Controller overhead is not negligible once the invocation interval falls below a few milliseconds. Finally, thermal state must be fed back into the policy rather than treated as a post-hoc constraint. These four claims are supported independently by [3], [5], [6], [9] and [12], and they are not controversial.

25.4 Where the evidence conflicts

The disagreements are more instructive. Reported savings range from single-digit percentages to figures above forty per cent, but the spread is almost entirely explained by differences in what is being normalised and what counts as the baseline. Papers that compare against a fixed maximum-performance governor report larger gains than papers that compare against an tuned heuristic. Papers that integrate energy over a fixed wall-clock interval report larger gains than papers that integrate until an equal amount of work has been served. Reference [5] is explicit that some of the dramatic results in this literature do not survive an implementation-overhead audit.

There is also a genuine disagreement about state. Several 2022-2024 works assume the observation is Markov and a single vector suffices. Reference [15] argues that on-device DVFS control is fundamentally partially observable and that a recurrent belief state is required. Reference [16] goes further and adds hierarchy. The project environment in [25] takes the first position: a five-element vector with no history. That is a defensible simplification for a course project, but it should be stated as a limitation rather than presented as a settled modelling choice.

A third disagreement concerns action granularity. Reference [21] compresses a large action space by partitioning it, while [7] and [16] expand it to include graph structure and hierarchy. The four discrete actions in [25] are at the compressed end of that spectrum. They are easy to export to C and easy to test, which is a real advantage for reproducibility, but they cannot express a per-cluster frequency choice. Any claim that the project performs joint cluster and frequency control should therefore be read as a statement about the interpretation of the four actions, not about hardware actuation.

25.5 Gaps a student repository can actually close

A course project cannot build a cycle-accurate simulator, fabricate a board, or reproduce a five-year research programme. It can, however, close four gaps that the literature repeatedly leaves open. The table below separates those gaps from the ones that must remain future work; this distinction is carried into the claim boundaries in Section 29.

Gap	What the literature leaves open	Feasible here?	Where addressed
Normalisation	Energy rarely divided by served work	Yes, in the C engine	Section 26
Reproducibility	Seeds and build steps often omitted	Yes, pinned and scripted	Section 27, App. A
Export verification	Python and C actors rarely compared	Yes, a 1,000-case parity test	Section 26
Thermal realism	Requires sensors and a board	No	Section 29
Hardware actuation	Requires kernel or firmware access	No	Section 29

The practical conclusion is that the repository should be judged on the first three gaps. It can demonstrate a correct, reproducible, work-normalised simulation and a verified export path. It cannot demonstrate energy savings on silicon, and the accompanying text should never imply otherwise.

The annotated bibliography beginning on page 36 gives the publication status and the role of each reference used in this synthesis.

26. Post-Review Remediation and Re-Measurement

26.1 What the original number actually measured

Section 20 reported that the pinned snapshot displayed a 92 per cent energy improvement. Compiling main.c from commit 2093bec and running it with a 15 W profile, an initial queue of 0.5 and 100 steps reproduces the figure exactly: the performance branch accumulates 1500.00 J and the learned branch 120.00 J. The derivation is transparent once the constants are visible. The performance branch applies action 3 on every tick, whose power ratio is 1.00, so it accumulates 100 by 15 W by 1.00 = 1500 J. The learned branch selects action 0 on every tick, whose ratio is 0.08, so it accumulates 100 by 15 W by 0.08 = 120 J. The reported improvement is 1 minus 0.08, that is 92.0 per cent.

Quantity	Performance branch	DRL branch	Ratio
Action applied each tick	3 (fixed)	0 (policy)	-
Assigned power ratio	1.00	0.08	0.08
Tick energy at 15 W	15.00 J	1.20 J	0.08
Cumulative over 100 ticks	1500.00 J	120.00 J	0.08
Reported improvement	-	-	92.0 per cent

The figure is therefore an arithmetic consequence of two constants chosen by the programmer. It is not a measurement, it is not a learned behaviour worth 92 per cent, and it does not depend on the workload except through the queue clipping described below. Recognising this is not a criticism of the students who built the prototype; it is exactly the kind of accounting error that [5] warns about, and finding it is a legitimate outcome of a literature review.

26.2 Three defects identified during inspection

- Uncounted work. The queue was clamped to the interval [0, 1] after each tick. Any excess arrival was silently discarded, so a policy that served slowly appeared to consume less energy while delivering less useful work. The dashboard had no field in which this loss could be seen.

- Unbuildable test suite. The inference function and the hardware step function were both compiled into main.c, which also defined main(). Linking any of the seven native test files against it failed with a duplicate-symbol error. A test suite that cannot be built provides no assurance at all.

- Non-portable parity check. The Python-C agreement script referenced a Windows executable name and could not run on the Linux host used for the review. The exported C actor was therefore never actually compared with the Python policy in the reviewed snapshot.

26.3 Changes implemented in the final revision

- The inference path was moved to inference.c and inference.h, and the simulated hardware model to hardware.c and hardware.h. main.c is now only a command-line driver. All seven native tests compile and pass under make test.

- Work accounting was added: arrivals, served work, dropped work and the number of clipping events are recorded for both branches. Served work is the smaller of the action service rate and the work actually available, which removes the possibility of serving work that does not exist.

- The driver reports joules per unit of served work and prints explicit warnings when served work is unequal or backlog clipping occurs. The parity script now takes a model path argument and runs on any platform via make parity.

26.4 Re-measurement on the revised engine

Both branches now receive the same pseudorandom arrival stream, seeded once per branch with srand(42). The deadline override and the power cap logic are retained so that the comparison remains recognisably the same experiment. Running ./scheduler 15 0.5 100 100 on the revised engine gives the following accounting.

Policy	Energy (J)	Served	Dropped	J per served
Performance governor	1500.00	26.306	0.000	57.0215
DRL C scheduler	199.20	26.199	0.000	7.6034
Ratio DRL / baseline	0.1328	0.9996	-	0.1333

A wider sweep was run with benchmark_equal_work.py over 20 hardware, queue and power-cap profiles at 200 steps each, with bootstrap 95 per cent confidence intervals from 10,000 resamples. The mean raw improvement was 86.702 per cent (CI 86.675 to 86.729) and the mean normalised improvement was 86.698 per cent (CI 86.671 to 86.725). The DRL-to-baseline served-work ratio was 0.99967 and no dropped work was recorded in any profile. The full machine-readable record is in results/equal_work_benchmark.json.

The first conclusion is that the earlier objection about unequal served work has been resolved. Both branches now complete essentially the same amount of work on the same arrival stream, and the raw and normalised improvements agree to within 0.01 percentage points. The second conclusion is less comfortable: removing the unfairness did not remove the large headline number. It fell from 92.0 to about 86.7 per cent because the revised policy occasionally selects actions 1 and 2 instead of action 0, but it remains large because power ratios 0.08 and 1.00 are still assumptions in the code.

26.5 Why the number is still not an energy saving

Energy in this program is the sum of a table lookup multiplied by a user-supplied wattage and a unit tick duration. No quantity in that expression is measured, calibrated, or derived from a processor model. Changing 0.08 to 0.008 would double the apparent saving without changing a single line of control logic. The number is a property of the model, and the only honest description of it is a simulated power ratio under an assumed profile. Section 29 formalises this as a claim boundary.

What the revised engine does establish is narrower but real: given an assumed power model, a work-normalised comparison is possible; the two branches can be made to serve equal work; and the result can be reproduced from a pinned commit with one command. Those are the properties that the review asked for in Section 22, and they are now verifiable.

26.6 Reproducing these figures

From a clean checkout, run make, make test, make benchmark, and python benchmark_equal_work.py --profiles 20 --steps 200 --seed 2026. The first command builds the engine, the second runs all seven native tests, the third prints the accounting table above, and the fourth writes the JSON summary. To compare the original behaviour, check out commit 2093bec and build main.c from that revision; the history is preserved in the current repository precisely so that this comparison remains possible.

Caution. The parity test verifies that the C actor agrees with the Python policy it was exported from. It does not verify that either policy is a good scheduler.

27. Statistical Protocol and Reporting Standards

27.1 A paired design, not two independent samples

Every comparison in the repository is paired. For each seed s, the learned policy and each baseline are evaluated on the same arrival sequence, the same initial queue, the same episode length and the same hardware profile. The unit of analysis is therefore the per-seed difference, not the difference between two independent means. Pairing removes the between-seed variance caused by unusually easy or unusually difficult workloads, which is often larger than the effect being measured.

A minimum of 30 seeds is used for any reported mean, following the convention already used in evaluate_model.py and compare_adaptive_policies.py. Seeds are consecutive integers from a published base rather than hand-picked values. The same seed set is used for every policy within a single table; if a policy fails to complete an episode, the entire seed is discarded for all policies so that the pairing is preserved.

27.2 Effect sizes and uncertainty

For a metric m and seed s, define d(s) = m_learned(s) - m_baseline(s). Report the mean of d, its standard deviation, and a 95 per cent bootstrap confidence interval built from 10,000 resamples of the seed-level differences with a fixed generator seed. Also report the standardised effect size, the mean of d divided by its standard deviation, because a statistically significant difference of 0.2 per cent in a simulated metric is not a meaningful engineering result. Where the outcome is a proportion, such as the fraction of episodes with no deadline misses, use a Wilson interval rather than a normal approximation.

27.3 Multiple comparisons

Comparing a learned policy against four fixed actions, a heuristic and two classical schedulers produces seven hypothesis tests per metric. At an uncorrected threshold of 0.05 the chance of at least one false positive is roughly 30 per cent. The repository therefore applies a Holm correction across the family of comparisons for each metric and reports both raw and adjusted results. Exploratory figures are labelled as exploratory; only the pre-registered primary metric, joules per unit of served work, is used for a confirmatory claim.

27.4 Reporting template

Field	Required content
Claim	One sentence, stating the population and the metric
Configuration	Commit hash, seed base, episode length, hardware profile
Substrate	Simulated, simulator, or physical board; evidence grade
Primary metric	Joules per unit of served work, with units
Baselines	Every comparator, with identical workloads confirmed
Uncertainty	Mean difference, sd, effect size, 95 per cent interval, n
Correction	Holm-adjusted result across the comparison family
Limitations	What the claim does not cover
Reproduction	The exact command that regenerates the numbers

27.5 Reproducibility checklist

- A pinned commit hash appears in every table caption.
- All random seeds are explicit and published in the results file.
- Library versions are recorded by the CI workflow on each run.
- Raw per-seed values are committed, not only the summary statistics.
- Every figure is generated by a script, never edited by hand.
- Negative and null results are retained rather than dropped.

These conventions are cheap to apply and they remove most of the ambiguity that makes cross-paper comparison in this field so difficult. They are also the reason the corrected figures in Section 26 can be checked by an examiner in under a minute.

28. Team Contributions and Project Governance

28.1 Project team

The work reported in this review and in the accompanying repository was carried out by the following five students. Contact details are included so that the contribution record can be verified if required.

#	Name	Registration	Institutional email
1	Aditya Rajeshwar Karn	24BEL0087	aditya.rajeshwar2024@vitstudent.ac.in
2	Yuvraj Singh	24BEL0057	yuvraj.singh2024@vitstudent.ac.in
3	Aditya Singh	24BEL0008	aditya.singh2024c@vitstudent.ac.in
4	Atharva Sivaraman	24BEL0082	atharva.sivaraman2024@vitstudent.ac.in
5	Yash Jain	24BEL0035	yash.jain2024a@vitstudent.ac.in

28.2 Contribution matrix

The division of work below reflects the areas each member was responsible for. It is recorded as a working agreement rather than as a legal assertion, and it should be confirmed by each member before final submission.

Area	Lead	Artefacts owned
Environment and training	Aditya Rajeshwar Karn	edge_env.py, train_model.py, seeds and hyperparameters
C inference and export	Yuvraj Singh	inference.c, weights.h, export_weights.py, parity test
Baselines and native tests	Aditya Singh	scheduler.c, task_queue.c, test_*.c, Makefile
Evaluation and statistics	Atharva Sivaraman	benchmark_equal_work.py, results/, work accounting
Review and documentation	Yash Jain	docs/, README.md, claim boundaries, bibliography

28.3 Workflow and internal review

The team used a single protected main branch with short-lived feature branches and pull requests. Each pull request required a passing CI run before merge; the workflow builds the C engine, executes all seven native tests, runs the benchmark and the equal-work harness, and checks Python syntax. Numerical changes were required to include the regenerated JSON results file so that reviewers could see the per-profile values rather than a screenshot of a chart. Review comments were resolved in-thread and the resolution recorded in the pull request.

The literature review was maintained as a living document. Each of Sections 5 to 17 was drafted by one member and checked by a second against the source paper, with disagreements resolved by returning to the original text. Sections 18 to 23 were written after the code inspection so that the implementation assessment could be separated from the published findings. The supplementary Sections 24 to 29 were added in the final revision, after the remediation described in Section 26 had been completed and re-measured.

28.4 Statement of originality

The code, the analysis and this document are the original work of the team, except for the openly licensed third-party libraries declared in requirements.txt and the published works cited in the annotated bibliography. No hardware measurement, third-party dataset, human-subject data or generative text service was used to produce the reported results. Where a figure from another work informed the design, it is cited at the point of use. The team accepts responsibility for the limitations stated in Section 29 and for the accuracy of the reproduction commands in Appendix A.

29. Limitations, Ethics, and Claim Boundaries

29.1 Model-level limitations

The power model is a four-entry lookup table. It has no dependence on voltage, frequency, capacitance, temperature, instruction mix or memory pressure, and it is not calibrated against any device. The service-rate table has the same character. Consequently the environment cannot exhibit the non-linearities that motivate most of the literature: no frequency-dependent leakage, no thermal throttling feedback, no shared-cache contention, and no transition latency when switching operating points. Reward shaping in edge_env.py uses hand-set coefficients, so the learned policy optimises a designer's intuition rather than a measured objective.

The observation is a five-element Markov vector with no history. Reference [15] argues that on-device control is partially observable, and [6] shows that temporal encoding changes behaviour materially. A policy trained here may therefore be exploiting regularities in a synthetic arrival process that would not exist on a real workload trace. Generalisation to unseen workload distributions has not been established.

29.2 Hardware and deployment limitations

The program does not call sched_setaffinity, does not write to CPU frequency governors, does not read RAPL or an external power meter, and does not run with real-time privileges. Deadline misses in the C scheduler are simulated ticks, not milliseconds on a board. The Streamlit dashboard compiles and executes the C simulator; it is a visualisation of a model, not a control panel for a machine. Any deployment claim would require a kernel-level integration, a measurement harness, thermal instrumentation and a safety review, none of which is present.

29.3 Ethical considerations

Overstating energy savings is the primary ethical risk in this area. Efficiency claims influence procurement and deployment decisions, and a simulated 87 per cent improvement presented as a measured result would be misleading. The team therefore labels every energy figure as simulated, publishes the model constants that produce it, and states the normalisation explicitly. The second risk is dual use: a scheduler that can starve a workload to reduce power can also be used to degrade a shared system deliberately. The work is released under a permissive licence for educational use, and the dashboards expose dropped work precisely so that starvation cannot be hidden.

The team also affirms the academic integrity of the submission. All sources are cited, the distinction between published findings and locally generated results is maintained throughout, and the original pinned snapshot is retained in version history so that the before-and-after comparison in Section 26 can be independently checked.

29.4 Claim-to-evidence table

Claim	Status	Evidence required to upgrade
PPO can be trained in this environment	Supported	Already demonstrated by train_model.py
C actor matches the exported Python actor	Supported after parity run	make parity with the matching archive
Energy per served work unit is measured	Not supported	Power meter and instrumented board
The policy saves 87 per cent energy	Not supported	Hardware A/B study with confidence intervals
Deadline guarantees are provided	Not supported	Real-time analysis and hardware timing
Generalises to unseen workloads	Not tested	Held-out traces and transfer study

29.5 Path to a defensible hardware study

A credible next step would replace the power table with measurements from a board such as a big.LITTLE development platform, using an external meter or an on-chip energy counter, and would calibrate the service-rate table from timed microbenchmarks at each available operating point. The same paired-seed protocol from Section 27 would then apply directly, with equal-work normalisation retained as the primary metric. Only after that calibration would a percentage saving carry physical meaning.

Appendix A. Reproducible Artefact Manifest

Every artefact named in this review is available in the repository at https://github.com/AdityaRezan/energy-aware-drl-scheduler. The commands below regenerate the numbers quoted in Sections 26 and 27 from a clean checkout. The baseline commit 2093bec is retained in the history of the same repository for before-and-after comparison.

Artefact	Purpose	Verification command
Makefile	Build and test entry points	make test
inference.c / weights.h	Exported C actor	make parity
hardware.c / hardware.h	Power, thermal and work model	make benchmark
main.c	Command-line benchmark driver	./scheduler 15 0.5 100 100
edge_env.py	Five-feature Gymnasium environment	python train_model.py
benchmark_equal_work.py	Paired profiles and bootstrap intervals	python benchmark_equal_work.py
test_*.c (7 files)	Native scheduler and inference tests	make test
test_policy_parity.py	1,000-observation Python-C agreement	make parity
results/*.json	Per-profile raw and summary values	committed with each run
docs/	This review, PDF and editable DOCX	-
TEAM.md	Contributor and governance record	-
.github/workflows/ci.yml	Build, test and benchmark on push	automatic

Appendix B. Glossary, Symbols, and Abbreviations

Term or symbol	Meaning in this document
DVFS	Dynamic voltage and frequency scaling
DRL / PPO / DQN	Deep reinforcement learning; proximal policy optimisation; deep Q-network
POMDP	Partially observable Markov decision process
SoC / DAG	System on chip; directed acyclic graph of dependent tasks
FCFS / RR	First-come first-served; round-robin scheduling
E(s), P(a)	Simulated energy at step s; assigned power ratio for action a
W(s), D(s)	Served work and dropped work at step s
J per served	Cumulative simulated joules divided by cumulative served work
d(s)	Per-seed paired difference, learned minus baseline
CI / sd	Confidence interval; standard deviation
Grade A to D	Evidence grading rubric defined in Section 24.3
Pinned snapshot	Commit 2093bec, inspected 16 October 2026
Action 0 to 3	Four discrete simulated operating points, lowest to highest power

End of supplementary material. The annotated references [1] to [25] follow on pages 36 to 38. Page numbering throughout this document includes the front matter and the annotated bibliography.

Annotated References

References 1–8 | IEEE-style numbered entries

[1] Y. G. Kim, M. Kim, J. Kong, and S. W. Chung, "An adaptive thermal management framework for heterogeneous multi-core processors," IEEE Transactions on Computers, vol. 69, no. 6, pp. 894–906, 2020. https://doi.org/10.1109/TC.2020.2970062

Role: hardware-based thermal migration and DVFS baseline.

[2] B. Salami, H. Noori, and M. Naghibzadeh, "Fairness-aware energy efficient scheduling on heterogeneous multi-core processors," IEEE Transactions on Computers, vol. 70, no. 1, pp. 72–82, 2021. https://doi.org/10.1109/TC.2020.2984607

Role: fairness-aware scheduling on heterogeneous CPU hardware.

[3] Y. Wang, W. Zhang, M. Hao, and Z. Wang, "Online power management for multi-cores: A reinforcement learning based approach," IEEE Transactions on Parallel and Distributed Systems, vol. 33, no. 4, pp. 751–764, 2022. https://doi.org/10.1109/TPDS.2021.3092270

Role: workload-phase learning, power capping, and uncore control; cluster context.

[4] M. Mohammadi and H. Beitollahi, "Q-scheduler: A temperature and energy-aware deep Q-learning technique to schedule tasks in real-time multiprocessor embedded systems," IET Computers & Digital Techniques, vol. 16, no. 4, pp. 125–140, 2022. https://doi.org/10.1049/cdt2.12044

Role: learned energy–thermal task dispatch; homogeneous multiprocessor scope.

[5] T. T. Sung and B. Ryu, "Deep reinforcement learning for system-on-chip: Myths and realities," IEEE Access, vol. 10, pp. 98048–98064, 2022. https://doi.org/10.1109/ACCESS.2022.3203401

Role: implementation overhead and the limits of transferring neural schedulers.

[6] T. Zhou and M. Lin, "CPU frequency scheduling of real-time applications on embedded devices with temporal encoding-based deep reinforcement learning," Journal of Systems Architecture, vol. 142, Art. no. 102955, 2023. https://doi.org/10.1016/j.sysarc.2023.102955

Role: deployed temporal DVFS and low-overhead inference for periodic edge tasks.

[7] C. Peng, M. Wang, J. Liu, L. Mo, and D. Niu, "Energy-efficient real-time DAG task scheduling on multicore platform by deep reinforcement learning," in 2024 IEEE/CIC International Conference on Communications in China (ICCC), 2024, pp. 597–602. https://doi.org/10.1109/ICCC62479.2024.10681954

Role: graph-aware actor–critic task mapping with frequency selection.

[8] X. Li, T. Zhou, H. Wang, and M. Lin, "Energy-efficient computation with DVFS using deep reinforcement learning for multi-task systems in edge computing," IEEE Transactions on Sustainable Computing, vol. 10, no. 6, pp. 1116–1127, Nov. 2025. Accessible manuscript: arXiv:2409.19434v3 (May 2025). https://doi.org/10.1109/TSUSC.2025.3593971

Role: multi-task, multi-deadline governor. Journal status verified; detailed analysis also uses the open manuscript.

Annotated References (continued)

References 9–16 | IEEE-style numbered entries

[9] J. Li et al., "FiDRL: Flexible invocation-based deep reinforcement learning for DVFS scheduling in embedded systems," IEEE Transactions on Computers, vol. 74, no. 1, pp. 71–85, 2025. https://doi.org/10.1109/TC.2024.3465933

Role: invocation intervals and the energy cost of running the learning agent.

[10] Y. Liu, H. Qu, S. Chen, and X. Feng, "Energy efficient task scheduling for heterogeneous multicore processors in edge computing," Scientific Reports, vol. 15, Art. no. 11819, 2025. https://doi.org/10.1038/s41598-025-92604-6

Role: integrated non-learning task prioritization, placement, and predictive DVFS.

[11] F. Kazemi, B. Barzegar, H. Motameni, and M. Yadollahzadeh-Tabari, "An energy-aware scheduling in DVFS-enabled heterogeneous edge computing environments," The Journal of Supercomputing, vol. 81, Art. no. 1078, 2025. https://doi.org/10.1007/s11227-025-07432-2

Role: search-based DAG scheduling followed by slack-aware frequency control.

[12] M. Farooq, A. Zafar, and A. Samad, "FR-EAHTS: Federated reinforcement learning for enhanced task scheduling with hierarchical load balancing and dynamic power adjustment in multi-core systems," Telecommunication Systems, vol. 88, Art. no. 48, 2025. https://doi.org/10.1007/s11235-025-01276-0

Role: complex federated learning integration; discussion limited to accessible publication evidence.

[13] H. Wang, "Integrating DVFS and task scheduling to improve energy efficiency for heterogeneous edge devices: A reinforcement learning approach," Master of Computer Science thesis, Dalhousie University, Halifax, Canada, Dec. 2025. Thesis, not a journal or conference paper. https://dal.scholaris.ca/server/api/core/bitstreams/e7007d9f-e208-4c0a-af2f-454f584fcce5/content

Role: directly relevant on-device joint cluster–operating-point controller. Master's thesis, not a journal article.

[14] M. Hussain and M. Lin, "Learning in real-world partially observable environments: Revisiting efficient deep Reinforcement Learning DVFS optimization for resource-constrained embedded devices," Journal of Systems Architecture, vol. 176, Art. no. 103797, July 2026. https://doi.org/10.1016/j.sysarc.2026.103797

Role: partially observable DVFS control; distinguish performance gains from energy savings.

[15] M. Pivezhandi, A. Saifullah, and A. Jannesari, "HiDVFS: Hierarchical multi-agent DVFS for real-time OpenMP DAG workloads," arXiv:2601.06425, 2026, version 2, revised July 8, 2026. Preprint; the inspected record states that it is under review. https://arxiv.org/abs/2601.06425v2

Role: 2026 hierarchical DAG scheduling and runtime screening. Preprint status retained.

[16] S. Shao, P. Ye, B. Zhao, and H. Lu, "Multi-objective reinforcement learning scheduling framework for heterogeneous AI chips: Energy-efficient and latency-aware edge-cloud collaboration," Journal of King Saud University Computer and Information Sciences, vol. 38, Art. no. 825, 2026. Published online Aug. 31, 2026. https://doi.org/10.1007/s44443-026-01248-7

Role: coordinated offloading, chip selection, and DVFS; broader than local CPU scheduling.

Annotated References (continued)

References 17–25 | IEEE-style numbered entries

[17] R. Hossain, A. Banu, and M. S. Arefin, "Cpu scheduling from classical heuristics to learning-based and multicore paradigms: A systematic review, comparative analysis, and research roadmap," Computing, vol. 108, Art. no. 142, 2026. Published online Sept. 2, 2026. https://doi.org/10.1007/s00607-026-01736-8

Role: current cross-paradigm survey; its reviewed literature extends through 2025.

[18] S. E. Arda et al., "DS3: A system-level domain-specific system-on-chip simulation framework," IEEE Transactions on Computers, vol. 69, no. 8, pp. 1248–1262, 2020. https://doi.org/10.1109/TC.2020.2986963

Role: heterogeneous SoC simulation and system-level experimental infrastructure.

[19] B. Salami, H. Noori, and M. Naghibzadeh, "Online energy-efficient fair scheduling for heterogeneous multi-cores considering shared resource contention," The Journal of Supercomputing, vol. 78, pp. 7729–7748, 2022. https://doi.org/10.1007/s11227-021-04159-8

Role: shared-resource contention, fairness, and hardware-based comparison.

[20] Q. Jiang, X. Xin, L. Yao, and B. Chen, "METSM: Multiobjective energy-efficient task scheduling model for an edge heterogeneous multiprocessor system," Future Generation Computer Systems, vol. 152, pp. 207–223, 2024. https://doi.org/10.1016/j.future.2023.10.024

Role: non-learning multiobjective optimization of sequence, placement, and DVFS.

[21] J. Li, X. Zhang, Z. Wei, J. Wei, and Z. Ji, "Energy-aware task scheduling optimization with deep reinforcement learning for large-scale heterogeneous systems," CCF Transactions on High Performance Computing, vol. 3, pp. 383–392, 2021. https://doi.org/10.1007/s42514-021-00083-8

Role: partitioned actions and representation compression in large heterogeneous systems.

[22] S. K. Panda, M. Lin, and T. Zhou, "Energy-efficient computation offloading with DVFS using deep reinforcement learning for time-critical IoT applications in edge computing," IEEE Internet of Things Journal, vol. 10, no. 8, pp. 6611–6621, 2023. https://doi.org/10.1109/JIOT.2022.3153399

Role: adjacent deadline-aware offloading; communication changes the energy boundary.

[23] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," Annual Review of Control, Robotics, and Autonomous Systems, vol. 5, pp. 411–444, 2022. https://doi.org/10.1146/annurev-control-042920-020211

Role: methodological distinctions among robustness, constraint enforcement, and certification.

[24] H. Hou, S. N. Agos Jawaddi, and A. Ismail, "Energy efficient task scheduling based on deep reinforcement learning in cloud environment: A specialized review," Future Generation Computer Systems, vol. 151, pp. 214–231, 2024. https://doi.org/10.1016/j.future.2023.10.002

Role: taxonomy of energy models, DRL components, rewards, and evaluation settings in cloud scheduling.

[25] AdityaRezan, ·energy-aware-drl-scheduler,· GitHub source-code repository, 2026. Snapshot: 2093bec5206c051cd4d806b2f1bd8a58634e42e1. Reviewed Oct. 16, 2026. Software artifact, not a research paper. https://github.com/AdityaRezan/energy-aware-drl-scheduler

Revised submission: 26 Oct 2026. Baseline snapshot 2093bec retained in repository history.

Role: supplied project; Python/C simulator, PPO archive, and dashboard. Static inspection and local C checks; no hardware or PPO reproduction claimed.